CSLA201/CSL214: **Data Structure and Program Design – II**

Lab Assignment (Announcement: 23rd October 2026)

Due Date: 2nd November 2026 (By 1:00 pm)

For Batch - R1

Notes

1. You can choose to do the assignment alone or in a group of 2 students. If done in a group, both students should be from the same practical-batch.
2. No late submission will be allowed.
3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
4. ***For this assignment, Tree data structure is required to be used. Hence, choose Trees (either AVL tree OR B-tree OR B+ tree) for any storage requirement you may have. Use of file operations to store/retrieve persistent data, would be given more credits.***
5. Evaluation will be done during 2nd November to 4th November 2026. Please make sure that you attend your lab hours during this week. For some batches/students, the evaluation may take place at scheduled time other than lab hours as will be announced.
6. You need to submit your source files by
   - Attaching these files in a mail by sending the mail to *dspd.assignment@gmail.com*

   Please attach only *.c, *.h files (and other i/p text files if any). *NO *.objs and *.exe to be attached.*
7. Please mark your subject line as *DSPD-2-Assignment-2: Your Enrollment numbers.*

Urban Company Service Management System

Urban Company connects customers with professionals who provide home services such as cleaning, plumbing, appliance repair, beauty services, and electrical work. Design a tree-based system (using ***either an AVL tree or B-tree or B+ tree*** as a data structure) to manage customers, professionals, services, and service requests while keeping the representation compact and the required operations efficient.

Data

The system maintains the following entities as records :

1. Customer

- Customer ID, Name, Pincode, Contact Number

2. Professional

- Professional ID, Name, Pincode, Contact Number, Availability status, Services provided by the professional

3. Service

- Service ID, Service Name, Base Price

4. Service Request

- Request ID, Customer, Service, Assigned Professional (if any), Requested Date and Time
- Status: Pending, Assigned, Completed, or Cancelled

A professional can provide multiple services, but **each professional can provide at most 5 different services**.

A customer may raise multiple service requests. A professional may handle multiple requests over time, but can handle only one request at a time.

Functions : Implement the following operations:

1. Register Customer

Add a new customer to the system.

2. Register Professional

Add a new professional. The professional initially provides one or more services and is marked as available.

3. Add Service

Add a new service with its base price.

4. Create Service Request

A customer requests a particular service.

- Verify that the customer and service exist.
- Search for an available professional who provides the requested service.
- Prefer a professional from the same pincode.
- If none is available in the same pincode, select an available professional from another pincode.
- If no suitable professional is available, keep the request pending.
- Once assigned, the professional becomes unavailable.

5. Complete Service

Complete an assigned request.

- Mark the request as completed.
- Make the professional available again.
- The newly available professional should be considered for an appropriate pending request, if one exists.

6. Cancel Service Request

Cancel a pending or assigned request.

- Mark the request as cancelled.
- If a professional was assigned, make the professional available again.
- The freed professional should be considered for an appropriate pending request.

7. Search Operations

Implement:

a. Given a Service ID, display all available professionals who provide that service.
b. Given a Professional ID, display all services provided by that professional.
c. Given a Customer ID, display all service requests made by that customer in chronological order.

8. Range Search Operation – Given two Customer IDs, C1 and C2, where C1 < C2, display all service requests made by customers having customer ID between C1 and C2, arranged in chronological order. Remove duplicates if any.

9. Sorting Operations

a. Service Demand:
Display all services in decreasing order of the number of service requests made for each service.

b. Professional Activity:
Display all professionals in decreasing order of the number of requests they have successfully completed.

Do not store these counts as additional fields in the nodes. Derive them from the representation maintained by your system.

10. Display Pending Requests

Display all currently pending requests in the order in which they were created.

11. Display Service History

Given a Professional ID, display the requests completed by that professional in chronological order.

Requirements and Constraints

1. The primary collections of **Customers, Professionals, Services, and Service Requests must be implemented using trees**.
2. **Do not create duplicate copies of an entity.** For example, there must be only one actual node representing a particular customer, professional, service, or request.
3. The maximum of **5 services per professional** is a property of the system and should be considered while designing the representation.
4. Do not store derived information such as *numberOfRequests*, *totalCompleted*, or *serviceDemand* as additional node fields. These values must be calculated from the maintained data.
5. Input data will not be provided in sorted order.
6. Initially sort the data based on IDs, for storage.
7. Your design should avoid unnecessary duplication of relationship information and should aim to minimize additional storage while keeping the required operations efficient.
8. Handle cases such as:
    - duplicate IDs,
    - invalid customer/service/professional IDs,
    - no available professional,
    - cancellation of an already completed/cancelled request,
    - completion of an unassigned request,
    - professional with no pending request after becoming available.
9. Implement the system using a menu-driven program and demonstrate all operations with suitable test data.